Rockset Distributed Query Execution Engine

Stateless Aggregator Compilation, Predicate Pushdown & CBO

< 3 Seconds	1024 Values
AGGREGATOR AUTOSCALING SPINUP (ZERO SHARD TRANSFER)	VECTORIZED SIMD BATCH EXECUTION ON LEAF NODES

1. The Analytical Query Concurrency Dilemma

Live operational applications require running analytical SQL queries—aggregations, multi-way joins, sub-selects—at high concurrency (thousands of QPS) with sub-50ms latencies. In monolithic architectures, complex queries exhaust CPU thread pools and lock shared storage caches.

2. Aggregator Query Compilation Pipeline

Rockset delegates query execution to an elastic fleet of stateless **Aggregators**:

The Aggregator Query Compilation Flow

[ANSI SQL] → [Lexer/Parser] → [Logical Operator Tree] → [Cost-Based Optimizer (CBO)] → [Predicate & Projection Pushdown] → [Physical DAG Broadcast to Leaves]

3. Cost-Based Optimization & Index Selection

The CBO leverages hyperloglog statistics from the Converged Index to select optimal execution paths:

Query Predicate Type	Chosen Execution Path	Performance Profile
High Selectivity (< 5%)	Pushdown to **Inverted Index**.	Extracts matching DocID Roaring Bitmaps in < 2ms.
Analytical Aggregation	Pushdown to **Columnar Index**.	SIMD AVX-512 vectorized scans on local NVMe chunks.
Entity Point Lookup	Pushdown to **Row Store**.	Direct $O(1)$ RocksDB prefix iterator seek.

4. Leaf-Level Vectorized Execution (SIMD)

Traditional Volcano iterator engines evaluate expressions one tuple at a time (`next()`), resulting in high virtual function call overhead and poor CPU cache locality. Rockset executes in **1024-element vectorized vector blocks** using C++ hardware intrinsics:

```
// Vectorized AVX-512 Columnar Filter Kernel
void filter_greater_than_simd(const float* vals, size_t count,
                              float threshold, uint64_t* out_bitmap) {
    __m512 thresh_vec = _mm512_set1_ps(threshold);
    for (size_t i = 0; i < count; i += 16) {
        __m512 chunk = _mm512_loadu_ps(&vals[i]);
        __mmask16 mask = _mm512_cmp_ps_mask(chunk, thresh_vec, _CMP_GT_OQ);
        out_bitmap[i / 64] |= ((uint64_t)mask << (i % 64));
    }
}
```

By executing directly over contiguous columnar memory buffers, the Leaf engine saturates memory bandwidth, filtering over **200 million rows per second per physical core**.

5. Distributed Join Coordination

Rockset executes distributed ANSI SQL joins using two dynamic strategies based on cost estimation:

1. Broadcast Hash Join (Fact × Dimension)

When joining a massive fact collection with a small dimension table (< 100MB), the Aggregator broadcasts the dimension table hash map to all Leaf nodes. Each Leaf executes the join locally against its resident fact shards with zero cross-network data shuffling.

2. Distributed Hash Shuffle Join (Fact × Fact)

When joining two massive collections, Leaves repartition and shuffle intermediate candidate streams across worker nodes using `hash(join_key) % K`, enabling unbounded horizontal join scaling.

6. On-Demand Horizontal Scaling of Aggregators

Because Aggregators maintain no shard replicas or local disk state, query compute scales elastically:

```
ScaleOut_Time = Boot_Container (≈ 2.5s) + Register_With_Coordinator (≈ 0.5s) = < 3.0 seconds
```

When query traffic surges from 50 QPS to 5,000 QPS, the cloud autoscaler launches 20 additional Aggregators. New nodes immediately pull DAG sub-queries and fan out to Leaves without data migration.

7. Empirical Query Latency vs. Concurrency

Query Workload Pattern	100 Concurrency (p95)	1,000 Concurrency (p95)	5,000 Concurrency (p95)
Point Lookups (DocID Seek)	1.2 ms	1.8 ms	2.4 ms
Filtered Scan (Inverted + Col)	18.4 ms	22.1 ms	26.8 ms
Multi-Way Join + Group By	42.5 ms	48.2 ms	54.1 ms

8. Architectural Best Practices & Guardrails

1. Pushdown Maximization
Always push filters and partial aggregations down to Leaf nodes. Streaming unreduced row batches across network sockets destroys Aggregator throughput.

2. Memory-Bounded Shuffle Buffers
Enforce strict spill-to-NVMe thresholds during distributed hash joins to prevent out-of-memory (OOM) crashes during data-skewed join key surges.